

بسم الله الرحمن الرحيم

آزمایش 8: طراحی، پیاده‌سازی و ارزیابی واحد واکنشی دستور (Instruction Fetch - IF) و رجیستر IF/ID در پردازنده Pipeline با استفاده از VHDL

اهداف آزمایش

در پایان این آزمایش انتظار می‌رود دانشجو بتواند:

- مفهوم معماری Pipeline و نحوه افزایش کارایی پردازنده از طریق همپوشانی اجرای دستورها را درک نماید.
- ساختار مرحله Instruction Fetch را در یک پردازنده Pipeline تحلیل کند.
- شمارنده برنامه (Program Counter) را به عنوان یک مدار ترتیبی طراحی و پیاده‌سازی نماید.
- حافظه دستورالعمل (Instruction Memory) را در قالب یک ROM مدل‌سازی کند.
- منطق انتخاب آدرس دستور بعدی را با استفاده از مالتی‌پلکسر طراحی نماید.
- مفاهیم (Stall)، Freeze، Branch و Flush را در سطح سخت‌افزار پیاده‌سازی کند.
- رجیستر IF/ID را طراحی و نقش آن را در هماهنگی مراحل Pipeline بررسی نماید.
- ساختار RTL واحد IF را تحلیل کرده و ارتباط بین مدارات ترتیبی و ترکیبی را درک کند.
- عملکرد مرحله IF را با استفاده از Testbench و Waveform ارزیابی نماید.
- رفتار Pipeline را در شرایط عادی، توقف (Stall) و پرش (Branch) تحلیل کند.

پیش‌نیازها

- آشنایی با فلیپ‌فلاپ‌ها و ثبات‌ها (Registers)
- آشنایی با مدارات ترتیبی و ترکیبی
- آشنایی با شمارنده‌ها
- آشنایی با حافظه‌های ROM و RAM
- آشنایی مقدماتی با معماری Pipeline
- آشنایی با مفاهیم Branch، Hazard و Jump
- تسلط بر دستورات Process در VHDL

توضیحات کلی

در پردازنده‌های مدرن، اجرای دستورات به صورت ترتیبی باعث کاهش کارایی سیستم می‌شود. برای افزایش سرعت اجرا از تکنیکی به نام Pipeline استفاده می‌شود که در آن مراحل مختلف اجرای دستورات به صورت همزمان روی چندین دستور انجام می‌شوند.

یک پردازنده Pipeline استاندارد معمولاً از پنج مرحله تشکیل شده است:

مرحله	نام
IF	Instruction Fetch
ID	Instruction Decode
EX	Execute
MEM	Memory Access
WB	Write Back

در این آزمایش تنها مرحله نخست یعنی Instruction Fetch (IF) طراحی و پیاده سازی می شود.

وظیفه این مرحله عبارت است از:

- نگهداری آدرس دستور جاری
- محاسبه آدرس دستور بعدی
- دسترسی به حافظه دستورالعمل
- واکنشی دستور
- مدیریت توقف های Pipeline
- مدیریت پرش ها و تغییر مسیر اجرای برنامه
- ارسال اطلاعات به مرحله Decode

آشنایی با عملکرد Pipeline

جدول زیر نحوه همپوشانی اجرای دستورات را در یک پردازنده Pipeline نشان می دهد.

Clock Cycle	IF	ID	EX	MEM	WB
1	I1	-	-	-	-
2	I2	I1	-	-	-
3	I3	I2	I1	-	-
4	I4	I3	I2	I1	-
5	I5	I4	I3	I2	I1

در هر سیکل کلاک، مرحله IF وظیفه تأمین دستور جدید برای Pipeline را برعهده دارد. بنابراین هرگونه خطا در این مرحله می تواند عملکرد کل پردازنده را مختل کند.

ساختار کلی واحد Instruction Fetch

واحد IF از چهار بخش اصلی تشکیل شده است:

1. Program Counter (PC): مهم‌ترین رجیستر در مرحله IF است. این رجیستر آدرس دستور جاری را نگهداری می‌کند.

برای مثال:

آدرس	دستور
0	ADD
1	SUB
2	AND
3	OR

اگر $PC = 2$ باشد، دستور AND از حافظه واکشی خواهد شد.

- منطق به‌روزرسانی PC:

- در حالت عادی برابر $Next_PC = PC + 1$ است که در این آزمایش هر خانه حافظه معادل یک دستور کامل فرض شده است؛ بنابراین افزایش آدرس برابر 1 در نظر گرفته می‌شود.
- در پردازنده‌های واقعی مانند MIPS و RISC-V برابر $Next_PC = PC + 4$ است زیرا هر دستور 32 بیت (4 بایت) فضا اشغال می‌کند.

2. حافظه دستورالعمل (Instruction Memory): حافظه دستورالعمل یک ROM است که کد ماشین برنامه در آن ذخیره شده است.

در این آزمایش:

- عمق حافظه = 16 خانه
- عرض هر خانه = 32 بیت

ساختار حافظه:

آدرس	محتوا
0	"X"01234567
1	"X"89ABCDEF
2	"X"11223344
3	"X"55667788
4	"X"99AABBCC
سایر خانه‌ها	"X"00000000

از آنجا که ROM دارای 16 خانه است، برای آدرس‌دهی از $PC(3 \text{ downto } 0)$ استفاده خواهد شد.

- **مفهوم Branch** : برخی دستورات باعث تغییر مسیر اجرای برنامه می‌شوند.

مثال:

BEQ R1,R2,LABEL

اگر شرط برقرار باشد:

$PC \leftarrow \text{Branch_Address}$

در غیر این صورت:

$PC \leftarrow PC + 1$

3. مالتی پلکسر انتخاب آدرس بعدی (Multiplexer): برای انتخاب بین آدرس متوالی و آدرس پرش از یک مالتی پلکسر استفاده می‌شود. سیگنال انتخاب Branch_Taken است.

- **مفهوم Hazard** : در پردازنده‌های Pipeline ممکن است تداخلاتی ایجاد شود که مانع اجرای صحیح دستورات شوند. این تداخلات Hazard نام دارند.
سه نوع اصلی Hazard عبارت‌اند از:

1. **Data Hazard** : وابستگی داده‌ای بین دو دستور
2. **Control Hazard** : ایجاد شده توسط Branch یا Jump
3. **Structural Hazard** : استفاده همزمان چند بخش از یک منبع سخت‌افزاری مشترک

- **مفهوم Freeze (Stall)** : برای رفع برخی Hazardها لازم است Pipeline موقتاً متوقف شود. در این حالت: $\text{Freeze} = 1$ خواهد بود.

هنگام فعال شدن Freeze:

- PC نباید تغییر کند.
 - رجیستر IF/ID نباید تغییر کند.
- بنابراین دستور جدیدی وارد Pipeline نمی‌شود.

- **مفهوم Flush**: پس از وقوع Branch ممکن است چند دستور اشتباه وارد Pipeline شده باشند. برای حذف این دستورات از Flush استفاده می‌شود.

هنگام فعال شدن '1' = Flush دستور ذخیره شده در رجیستر IF/ID به NOP تبدیل می‌شود.

- دستور NOP: مخفف عبارت No Operation است و این دستور هیچ عملیاتی انجام نمی دهد.

در این آزمایش:

"X"00000000

به عنوان NOP در نظر گرفته می شود.

4. رجیستر IF/ID: این رجیستر بین مرحله IF و ID قرار می گیرد.

وظایف آن:

- ذخیره دستور واکنشی شده
- نگهداری اطلاعات تا پایان سیکل جاری
- جلوگیری از تداخل مراحل مختلف Pipeline
- پشتیبانی از Freeze
- پشتیبانی از Flush

مراحل انجام آزمایش

۱. طراحی Program Counter :

یک رجیستر 8 بیتی طراحی کنید که دارای:

- clk
- rst
- freeze

باشد.

شرایط عملکرد:

- در صورت فعال شدن rst مقدار PC صفر شود.
- در صورت فعال شدن freeze مقدار PC ثابت بماند.
- در غیر این صورت مقدار PC با Next_PC به روزرسانی شود.

۲. طراحی مالتی پلکسر Branch

یک مالتی پلکسر 2 به 1 طراحی نمایید.

ورودی ها:

- PC + 1
- Branch_Address

خروجی:

- Next_PC

۳. طراحی Instruction Memory

یک ROM شامل 16 خانه 32 بیتی طراحی نمایید.

چند دستور نمونه در حافظه قرار دهید.

۴. طراحی رجیستر IF/ID

یک رجیستر 32 بیتی طراحی کنید که:

- در لبه بالارونده کلاک مقدار دستور را ذخیره کند.
- در صورت فعال شدن Freeze مقدار خود را حفظ نماید.
- در صورت فعال شدن Flush مقدار NOP را بارگذاری کند.

۵. اتصال بلوک‌ها و طراحی ماژول نهایی

تمام زیرمدارهای طراحی شده را در قالب یک Top Module به یکدیگر متصل نمایید.

۶. ایجاد پروژه و سنتز مدار

- یک پروژه جدید در Xilinx ISE یا Vivado ایجاد کنید.
- فایل‌های VHDL را به پروژه اضافه نمایید.
- مراحل Compile و Synthesis را اجرا کنید.
- خطاهای احتمالی را اصلاح نمایید.
- RTL Viewer را باز کنید.

پس از سنتز انتظار می‌رود اجزای زیر در RTL مشاهده شوند:

- رجیستر 8 بیتی PC
- جمع‌کننده PC+1
- مالتی‌پلکسر Branch
- حافظه ROM
- رجیستر IF/ID
- منطق Flush و Freeze

۷. نوشتن Testbench

سناریوهای زیر را پیاده‌سازی نمایید:

- تولید کلاک 10ns
- اعمال Reset
- اجرای عادی چند سیکل متوالی
- فعال‌سازی Freeze
- غیرفعال‌سازی Freeze
- اعمال Branch
- اعمال Flush
- بررسی عملکرد IF/ID

۸. شبیه‌سازی و مشاهده Waveform

سیگنال‌های زیر را بررسی نمایید:

- clk
- rst
- pc
- next_pc
- branch_taken
- branch_address
- freeze
- flush
- instruction
- ifid_instruction

از Waveform اسکرین‌شات تهیه نمایید.

9. تحلیل نتایج

در گزارش به پرسش‌های زیر پاسخ دهید:

1. چرا مرحله IF ترکیبی از مدارات ترتیبی و ترکیبی است؟
2. چرا هنگام Stall باید هم PC و هم IF/ID متوقف شوند؟
3. اگر فقط PC متوقف شود چه اتفاقی رخ خواهد داد؟
4. نقش رجیستر IF/ID در عملکرد صحیح Pipeline چیست؟
5. تفاوت Flush و Freeze چیست؟
6. چرا پس از وقوع Branch نیاز به Flush داریم؟
7. چرا در این آزمایش از PC+1 و نه PC+4 استفاده شده است؟
8. اگر حافظه دستورالعمل به صورت Synchronous ROM طراحی شود چه تغییری در Timing سیستم رخ خواهد داد؟
9. استفاده از NOP چه تأثیری بر CPI پردازنده خواهد داشت؟

الزامات گزارش نهایی

گزارش نهایی باید به صورت فایل PDF و شامل موارد زیر باشد:

- کد کامل VHDL مدار اصلی
- کد کامل Testbench
- اسکرین‌شات از:
 - کدهای نوشته شده
 - RTL Schematic
 - سنتز موفق
 - Waveform شبیه‌سازی
- پاسخ کامل سوالات تحلیل نتایج
- نتیجه‌گیری نهایی آزمایش

نحوه ارسال

فایل نهایی باید در قالب ZIP و با فرمت زیر ارسال شود:

StudentID_Name_CAL_HW8.zip